Task 1:

Process Synchronization and Child Process Monitoring Using waitpid()

```
root@Tejasri:~# nano sTask1.c
root@Tejasri:~#
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    pid_t pid, result;
    int status;
    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        sleep(2);
        printf("Child process completed.\n");
        exit(5);
    }
    printf("Parent process. PID = %d\n", getpid());
    printf("Waiting for child process...\n");
    result = waitpid(pid, &status, 0);
    if (result == -1) {
        perror("waitpid");
       return 1;
    }
    if (WIFEXITED(status)) {
        printf("Child exited normally with status = %d\n",
               WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("Child terminated by signal = %d\n",
              WTERMSIG(status));
    }
```

OUTPUT:

```
root@Tejasri:~# nano sTask1.c
root@Tejasri:~# gcc sTask1.c -o sTask1
root@Tejasri:~# ./sTask1
Parent process. PID = 564
Waiting for child process...
Child process started. PID = 565
Child process completed.
Child exited normally with status = 5
Parent process completed.
root@Tejasri:~#
```

Task 2:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Function	Purpose
getenv()	Retrieves the PATH environment variable
strtok()	Splits PATH into individual directories
access()	Checks file existence and execute permission
snprintf()	Builds the complete executable path
strdup()	Creates a modifiable copy of PATH
free()	Releases allocated memory

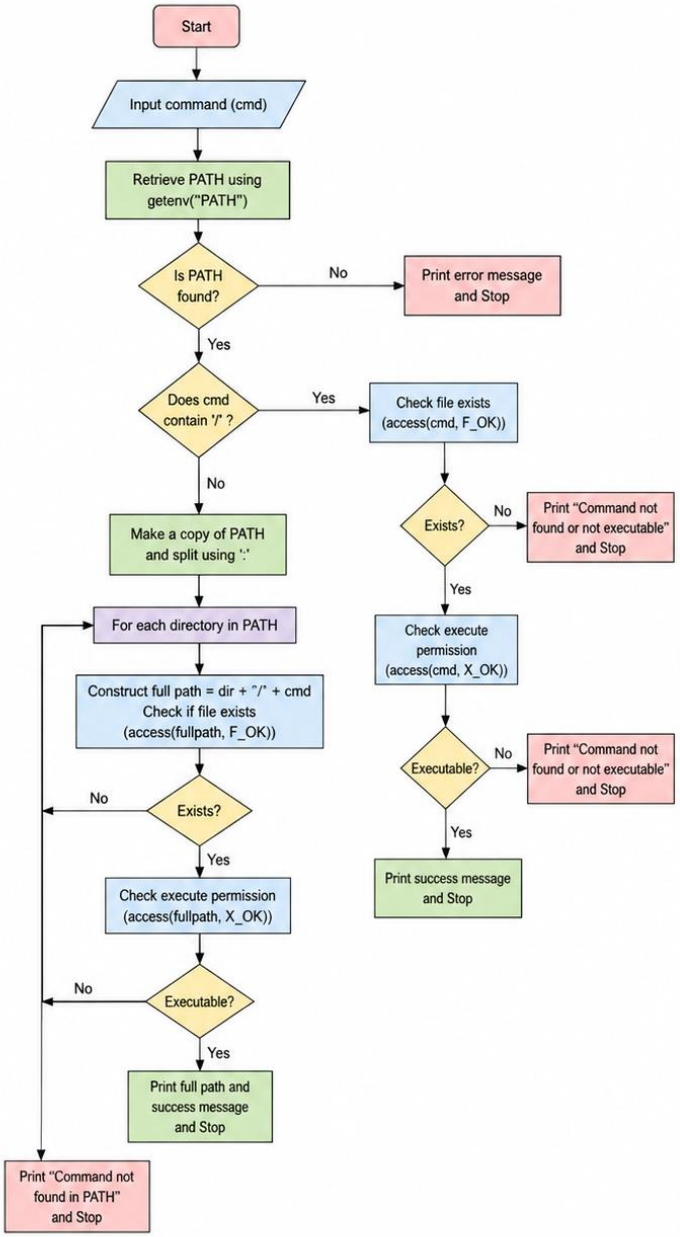
Flow Chart:

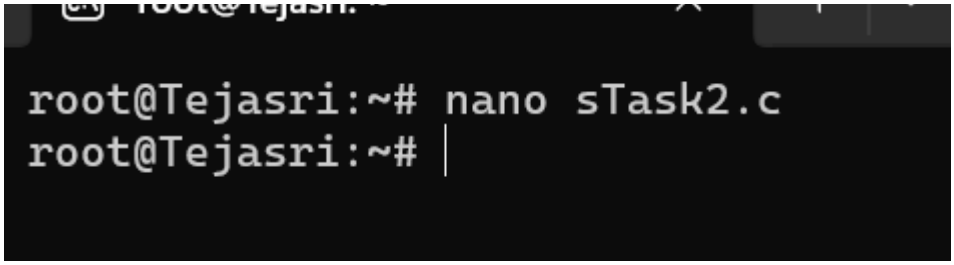
ALGORITHM

1. Start
2. Input command name from user (cmd).
3. Retrieve the PATH environment variable using getenv("PATH").

   If PATH is not found, print error message and stop.
4. If cmd contains '/' (i.e., user gave a path):
   - 4.1. Check if the file exists using access(cmd, F_OK).
   - 4.2. If it exists, check execute permission using access(cmd, X_OK).
   - 4.3. If executable, print success message.
   - 4.4. Else, print "Command not found or not executable".
   - 4.5. Stop.
5. Else (cmd does not contain '/'):
   - 5.1. Make a copy of PATH.
   - 5.2. Split the PATH using ':' as delimiter to get directories.
   - 5.3. For each directory in PATH do
     - 5.3.1. Construct full path = directory + '/' + cmd.
     - 5.3.2. Check if file exists at full path using access(fullpath, F_OK).
     - 5.3.3. If file exists, check execute permission using access(fullpath, X_OK).
     - 5.3.4. If executable, print full path and success message. Go to Step 6.
     - 5.3.5. Else, continue with next directory.
   - 5.4. If all directories are checked and command not found, print "Command not found in PATH".
6. Stop.

FLOWCHART





```
 GNU nano 7.2                                                    sTask2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
int main(int argc, char *argv[]) {
    char *path;
    char *path_copy;
    char *dir;
    char fullpath[PATH_MAX];
    int found = 0;
    if (argc != 2) {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }
    path = getenv("PATH");
    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }
    printf("PATH = %s\n\n", path);
    path_copy = strdup(path);
    if (path_copy == NULL) {
        perror("strdup");
        return 1;
    }
    dir = strtok(path_copy, ":");
    while (dir != NULL) {
        snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, argv[1]);
        if (access(fullpath, F_OK) == 0) {
            printf("File found: %s\n", fullpath);
            if (access(fullpath, X_OK) == 0) {
```

```
    dir = strtok(path_copy, ":");
    while (dir != NULL) {
        snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, argv[1]);
        if (access(fullpath, F_OK) == 0) {
            printf("File found: %s\n", fullpath);
            if (access(fullpath, X_OK) == 0) {
                printf("Execute permission: YES\n");
                printf("Command resolved successfully.\n");
                found = 1;
                break;
            } else {
                printf("Execute permission: NO\n");
            }
        }
        dir = strtok(NULL, ":");
    }
    if (!found) {
        printf("Command '%s' not found in PATH.\n", argv[1]);
    }
    free(path_copy);
    return 0;
}
```

OUTPUT:

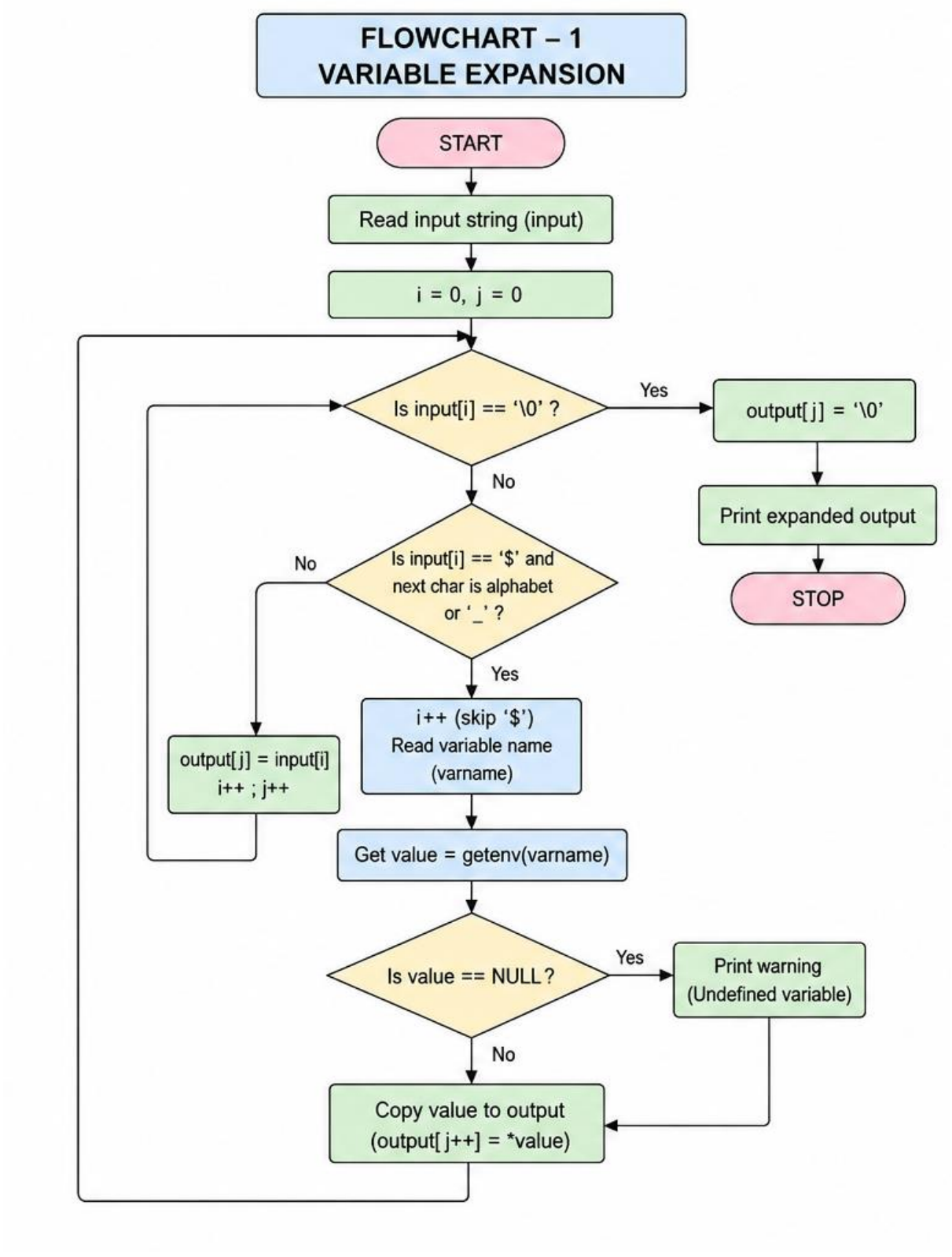
```
root@Tejasri:~# ./sTask2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files
/Java/jdk-21/bin:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System3
2/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0/:/mnt/c/WINDOWS/System32/OpenSSH/:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/P
rogram Files/nodejs/:/mnt/c/Program Files/MySQL/MySQL Shell 8.0/bin/:/mnt/c/Users/tejas/AppData/Local/Microsoft/WindowsApps:/mnt/c/Us
ers/tejas/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/c/Users/tejas/AppData/Roaming/npm:/snap/bin

File found: /usr/bin/pwd
Execute permission: YES
Command resolved successfully.
root@Tejasri:~#
```

Task 3:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

Function	Purpose
getenv()	Retrieves environment-variable values
fgets()	Reads input
strcspn()	Removes newline
isalpha()	Checks alphabetic character
isalnum()	Checks alphanumeric character

Flow Chart:




```
root@Tejasri:~# nano sTask3.c
root@Tejasri:~#
```

```
  GNU nano 7.2                                                  sTask3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX 1024
void expand_variables(char *input, char *output) {
    int i = 0;
    int j = 0;
    while (input[i] != '\0' && j < MAX - 1) {
        if (input[i] == '$') {
            char variable[100];
            int k = 0;
            int start = i;
            i++;
            if (input[i] == '{') {
                i++;
                while (input[i] != '\0' && input[i] != '}' &&
                       k < 99) {
                    variable[k++] = input[i++];
                }
                if (input[i] == '}')
                    i++;
            } else if (isalpha((unsigned char)input[i]) ||
                       input[i] == '_') {

                while ((isalnum((unsigned char)input[i]) ||
                        input[i] == '_') && k < 99) {
                    variable[k++] = input[i++];
                }
```

```
 GNU nano 7.2                                                    sTask3.c *
            } else {
                printf("Warning: Undefined variable: %s\n", variable);
            }
        } else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main() {
    char input[MAX];
    char output[MAX];

    printf("Enter a string: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;

    input[strcspn(input, "\n")] = '\0';

    expand_variables(input, output);

    printf("Original : %s\n", input);
    printf("Expanded : %s\n", output);

    return 0;
}
```

OUTPUT:

```
root@Tejasri:~# nano sTask3.c
root@Tejasri:~# gcc sTask3.c -o sTask3
root@Tejasri:~# ./sTask3
Enter a string: Tejasri
Original : Tejasri
Expanded : Tejasri
root@Tejasri:~#
```

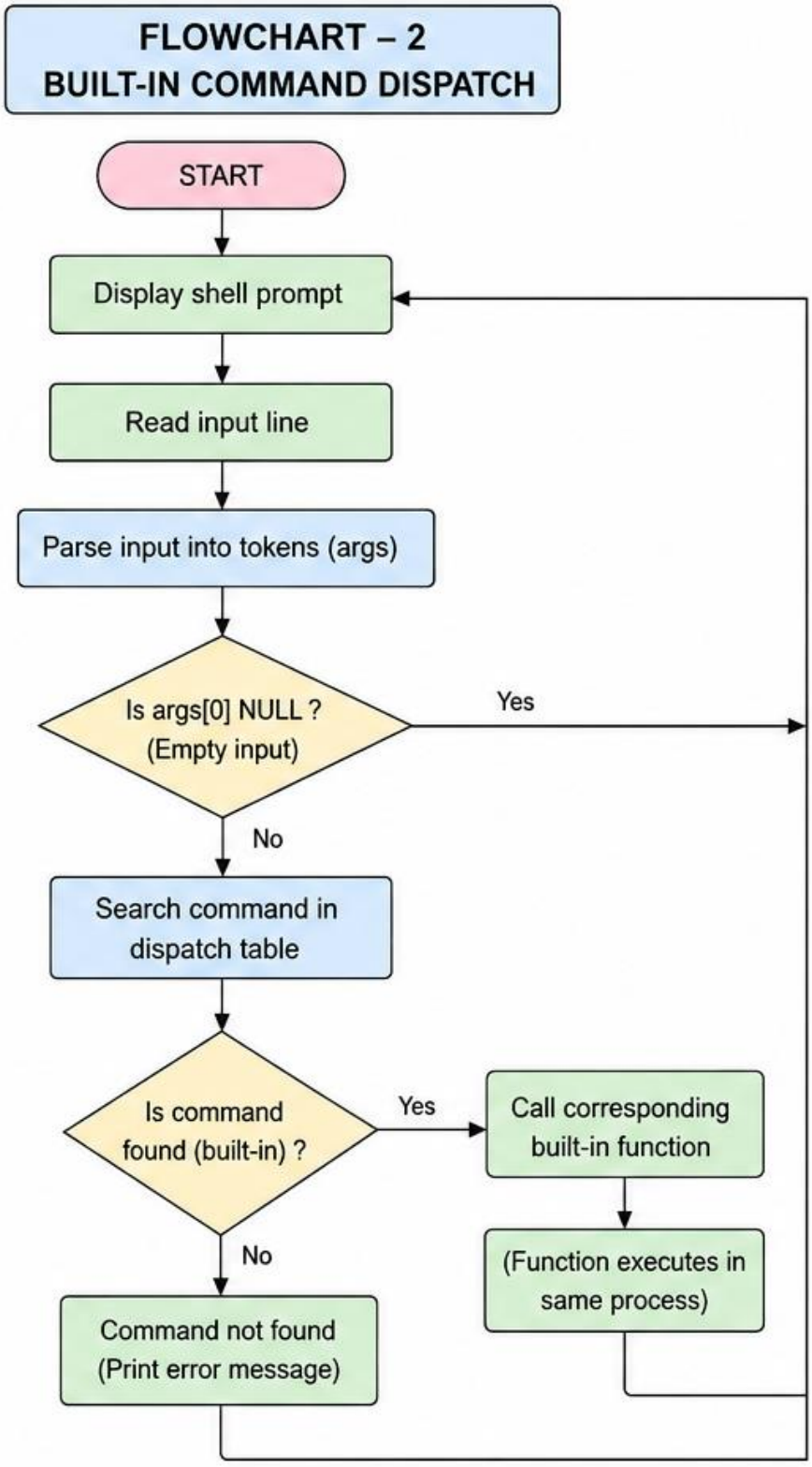
Task 4

To Identify Built-ins, Create Dispatch Table, Execute In-Process Built-ins, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Function	Purpose
chdir()	Changes the current working directory

Function	Purpose
getcwd()	Gets the current working directory
getenv()	Retrieves environment variables
exit()	Terminates the process
strcmp()	Compares command names
strtok()	Separates command into tokens

Flow Chart:


FLOWCHART – 2
BUILT-IN COMMAND DISPATCH
START
Display shell prompt
Read input line
Parse input into tokens (args)
Is args[0] NULL ?
(Empty input)
Yes
No
Search command in
dispatch table
Is command
found (built-in) ?
Yes
Call corresponding
built-in function
(Function executes in
same process)
No
Command not found
(Print error message)

```
root@Tejasri:~# nano sTask4.c
root@Tejasri:~#
```

```
 GNU nano 7.2                                                    sT
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100
void builtin_pwd(char **args) {
    char cwd[MAX_INPUT];
    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}
void builtin_cd(char **args) {
    if (args[1] == NULL) {
        char *home = getenv("HOME");
        if (home != NULL)
            chdir(home);
        else
            printf("HOME not set\n");
    } else {
        if (chdir(args[1]) != 0)
            perror("cd");
    }
}
```

```
GNU nano 7.2
}
void builtin_cd(char **args) {
    if (args[1] == NULL) {
        char *home = getenv("HOME");
        if (home != NULL)
            chdir(home);
        else
            printf("HOME not set\n");
    } else {
        if (chdir(args[1]) != 0)
            perror("cd");
    }
}
void builtin_echo(char **args) {
    int i = 1;
    while (args[i] != NULL) {
        printf("%s", args[i]);
        if (args[i + 1] != NULL)
            printf(" ");
        i++;
    }
    printf("\n");
}
void execute_command(char **args) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }
```

```c
    char *args[MAX_ARGS];
    char *token;
    while (1) {
        printf("myshell> ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        int argc = 0;
        token = strtok(input, " ");
        while (token != NULL && argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }
        args[argc] = NULL;
        if (strcmp(args[0], "exit") == 0) {
            printf("Exiting shell...\n");
            exit(0);
        }
        if (strcmp(args[0], "pwd") == 0) {
            builtin_pwd(args);
        } else if (strcmp(args[0], "cd") == 0) {
            builtin_cd(args);
        } else if (strcmp(args[0], "echo") == 0) {
            builtin_echo(args);
        } else {
            execute_command(args);
```

OUTPUT:

```
root@Tejasri:~# nano sTask4.c
root@Tejasri:~# gcc sTask4.c -o sTask4
root@Tejasri:~# ./sTask4
myshell> pwd
/root
myshell> echo Hello World
Hello World
myshell> abc
Command not found: abc
myshell>
```